

STL IN C++

STL → Standard Template Library

STL is not a part of Cpp. Programmer introduces it to make the Cpp more stronger

* Pair

Syntax : `Pair < Type1, Type2 > name-of-pair;`

Rohit, 30 → If we want to store this data we can do with the help of pair

`Pair < string, int > P;`

1) `P = make_pair("Rohit", 30)`

2) `P.first = "Rohit";`
`P.second = 30;`

Implementation:

Class is used to implement pair

Can we store more than two values in pair?

Yes, we can nested it

Suppose we want to store; name, age, weight

`pair < pair < string, int >, int > p`

`p = make_pair(make_pair("Damon", 25), 75);`

`cout << p.first.first << " " << p.first.second << " "`
`<< p.second << endl;`

name, age, weight
① ②

```
pair<string, pair<int, int>> p;
```

```
p = make_pair("Demon", make_pair(25, 75));
```

```
cout << p.first << " " << p.second.first << " " <<  
p.second.second << endl;
```

Like this we can store multiple values

* List

Just as a simple linked list but doubly

Syntax: `list<Type> name-of-list;`

```
list<int> l1;
```

```
l1.push-back(30);
```

```
l1.push-back(50);
```

```
l1.push-front(20);
```

30

30 ↔ 50

20 ↔ 30 ↔ 50

How we can iterate over the list?

```
for (auto it = l1.begin(); it != l1.end(); it++)  
{
```

```
    cout << *it << " ";
```

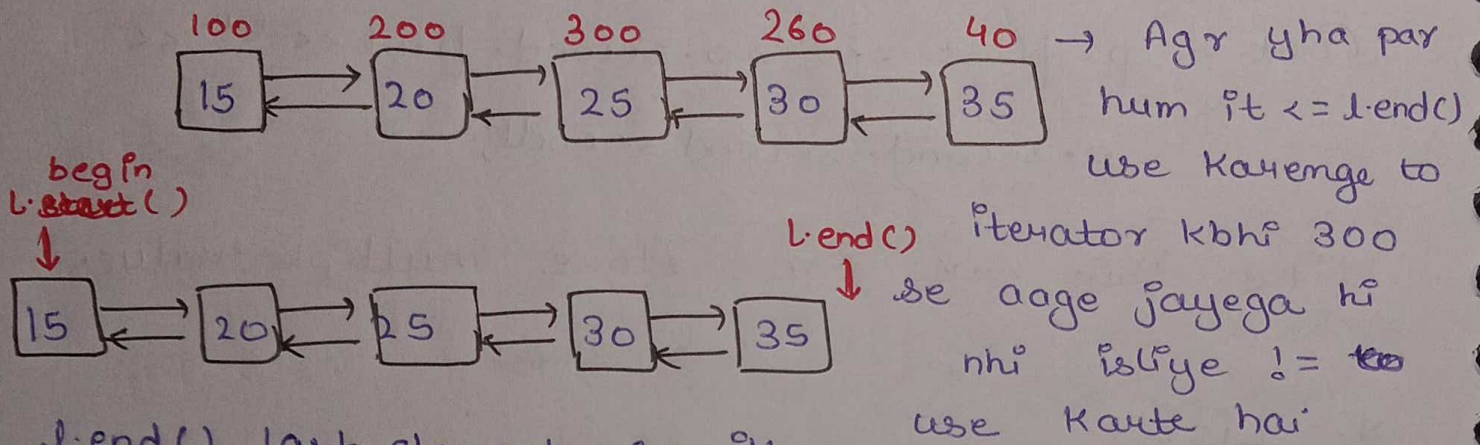
```
}
```

auto → we use this keyword to automatically determine whether it is integer, string etc.
datatype

auto == list<int>::iterator

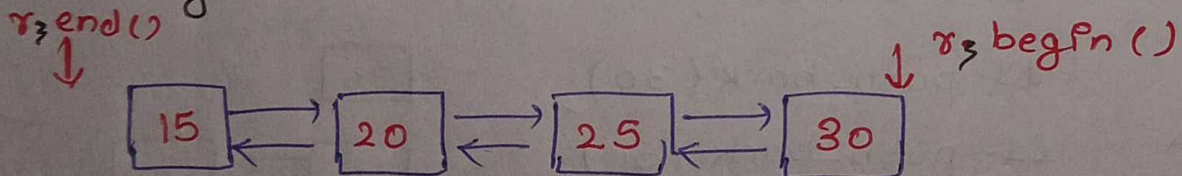
it != l.end() ✓ it <= l.end() ✗

We take != because some time iterator can point to address



$l.end()$ last element se ik step aage point kar rha hota hai

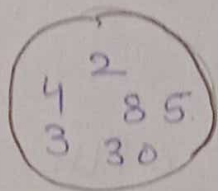
Printing the list in Reverse Order -



```
for (auto it = l.rbegin(); it != l.rend();  
     it++)  
{  
    cout << *it;  
}
```


* Set

- It is like a container
- It stores only unique element
- It stores value in sorted order



Set

If we print the element from set then it will be printed in sorted order..

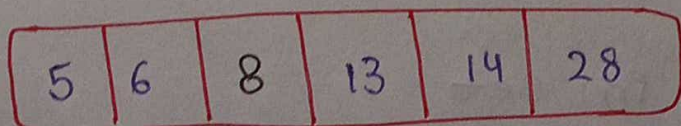
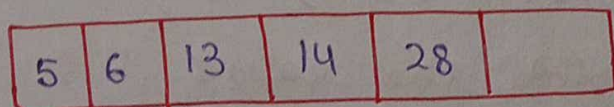
2 3 4 5 8 30

Example: Instagram id's (If you ever notice that

sometime when you create user account and when you enter username then it shows this account is already exist so here set is use to store the unique element or user ids)

→ In set we can insert, delete, search element in order of $\log n$ time ~~$O(n)$~~ $O(\log n)$

Implementation: If we think that implementation is done by Array then we are wrong because insertion in array can take $O(n)$ time.



Insert - 8
With the help of binary search it take $O(\log n)$ in delete and search but insert take $O(n)$ time

So the implementation is done by AVL Tree

AVL tree take $O(\log n)$ for insertion and same for deletion, search.

This can also be implemented with the help of "Red-Black Tree" because the rotation are less than "AVL Tree"

Syntax : `set<int> s;`

```
s.insert(10);
s.insert(20);
s.insert(40);
s.insert(10);
s.insert(45);

for (auto it = s.begin();
     it != s.end(); it++)
{
    cout << *it << endl;
}
```

It will print only unique value because it stores only unique value.

Now if we want that set should store the value in descending order

`set<int, greater<int>> s;`

After this we print the value then it will be printed in descending order.

Searching element in set

→ with the help of find function.


```
if (s.find(200) != s.end())
```

```
    cout << "Present";
```

```
else
```

```
    cout << "Absent";
```

This find function
will return the
iterator which point
to value

Another method to find element

```
if (s.count(210))
```

```
    cout << "Present"
```

```
else
```

```
    cout << "Absent"
```

OR

```
cout << s.count(280) << endl;
```

```
delete → s.erase(110);
```

We know it is a set
so element will be present
only one time so if it is
1 then present will be
answer if it 0 then
absent will be answer

Can we create a set of any class?

```
Class Person {
```

```
    public:
```

```
    int age;
```

```
    string name;
```

```
    bool operator < (const
```

```
    Person &other) const {
```

```
        return age < other.age;
```

```
    }
```

```
}
```

```
set < Person > s;
```

```
Person p1, p2;
```

```
p1.age = 10, p1.name = "Elena"
```

```
p2.age = 30, p2.name = "Damon"
```

```
s.insert(p1);
```

```
s.insert(p2);
```

If we run the above code
then it will give an error
because it doesn't ^{know} on
which basis it should sort
the element so we write
the code in class to handle
this case.

```

for (auto it = s.begin(); it != s.end(); it++)
{
    cout << it->age << " " << it->name << endl;
}

```

* Multiset

- It can store duplicate element in set.
- Element will be in sorted order

```
multiset<int> s;
```

```
s.insert(10);
```

```
s.insert(20);
```

```
s.insert(10);
```

```
s.insert(30);
```

```
for (auto it = s.begin(); it !=
    s.end(); it++)
```

```
{
```

```
    cout << *it << " ";
}
```

Output: 10 10 20 30

If we perform delete operation -

```
s.erase(10);
```

It will delete all the 10 which are present in the set.

* Unordered set

- It store unique element.
- It store data in unordered way

Search, Insert, delete

$O(1)$

Implementation
is done by
Hashing

Syntax : `unordered_set<int> s;`] store only unique element

unordered_multiset<int> s;] Store duplicate element as well

* Map

→ It store element in Key - value pair

↓ ↓
Unique duplicate

Index $\rightarrow$ 0 1 2 3 4 5 6

value	3	10	20	10	3	20	10
-------	---	----	----	----	---	----	----

If we count the number of occurrence of 10

But with map it take $O(\log n)$ then normally it will take $O(n)$

Key → 3 10 30

value → 2 3 2

2	3	2
---	---	---

Key will be choosen according to question.

`cout << m[10];` it will give ans - 3

→ Map store the value in sorted form on the basis of key

→ Search, insert, delete take $O(\log n)$ time
so this is implemented by AVL Tree or
Red-Black Tree else it giving the properties
of array.

Example: Instagram id, password

→ Key will be unique always

Syntax: `map <int, int> m;`

```
m.insert (make_pair (20, 30));  
m.insert (make_pair (30, 310));  
m.insert (make_pair (40, 230));  
m.insert (make_pair (20, 230));  
m.insert (make_pair (50, 30));
```

Its output will be :

20 30 30 310 40 230 50 30

Print only unique element

`m[100] = 60;`
`m[20] = 70;` } This type of method will
insert and update the element
if the key is already

Output:

20 70 30 310 40 230 50 30 100 60

Update the value
because key-20 is already
present

Insert at
last

```
cout << m[100];    // 60
```

```
cout << m[200];    // 0
```

Ab yeh key 200 present
nhi hai islye 0 ans mai aaya but aisa
bhi toh ho skta ki 200 key ki value hi 0
ho

```
if (m.count(200))  
    cout << m[200] << " ";
```

✓ This will check the
count if it is one then
the condition execute

* Multimap $\rightarrow$ Can store multiple value with same Key

`multimap <int, int> m;`

`m.insert (make_pair (20, 30));`

`m.insert (make_pair (30, 310));`

`m[100] = 60;`

`m[20] = 70;`

$\rightarrow$ We can't do this in multimap because

multiple key can have value 100 so it will throw an error

* Unordered Map

$\rightarrow$ Unique Keys are present

$\rightarrow$ but it can follow any order

Syntax: `unordered_map <int, int> m;`

Implementation: Hashing

Search, insert, deletion take $O(1)$ time

* Unordered_multimap

$\rightarrow$ Duplicate Keys are present

$\rightarrow$ Order doesn't matter

Syntax: `unordered_multimap <int, int> m;`

Implementation: Hashing

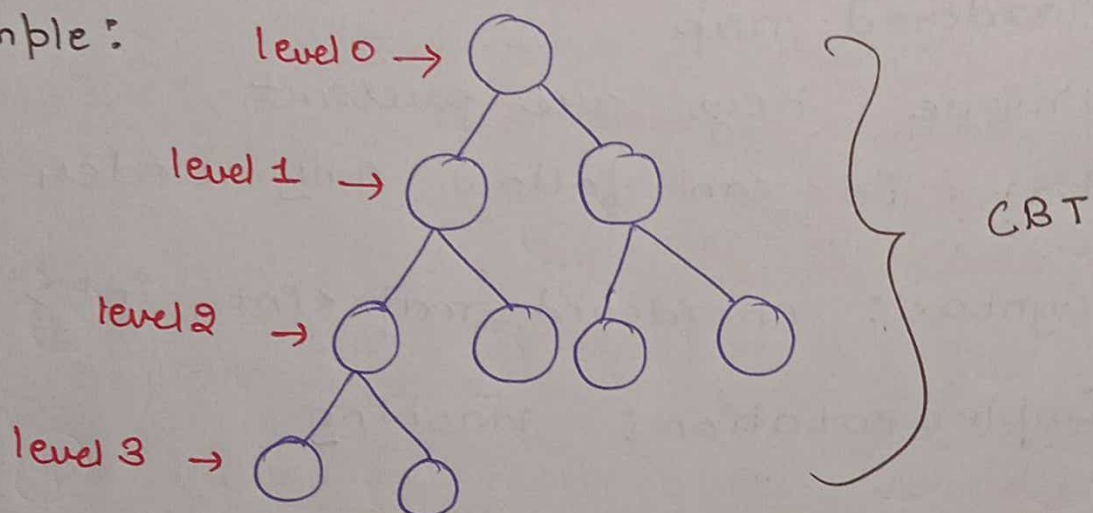
Take $O(1)$ time for an operation.

INTRODUCTION To HEAP DATA STRUCTURE

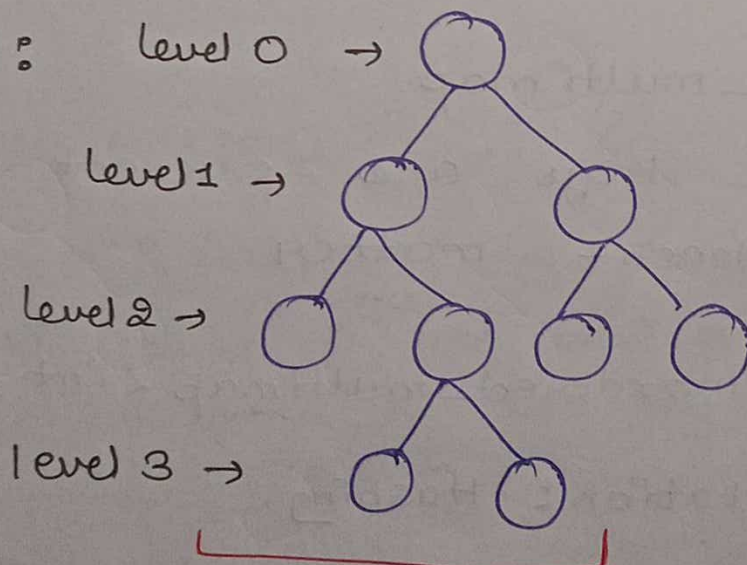
Heap : It is a Complete Binary Tree

- All level are completely filled except the last level
- At last level, nodes should be filled from left side

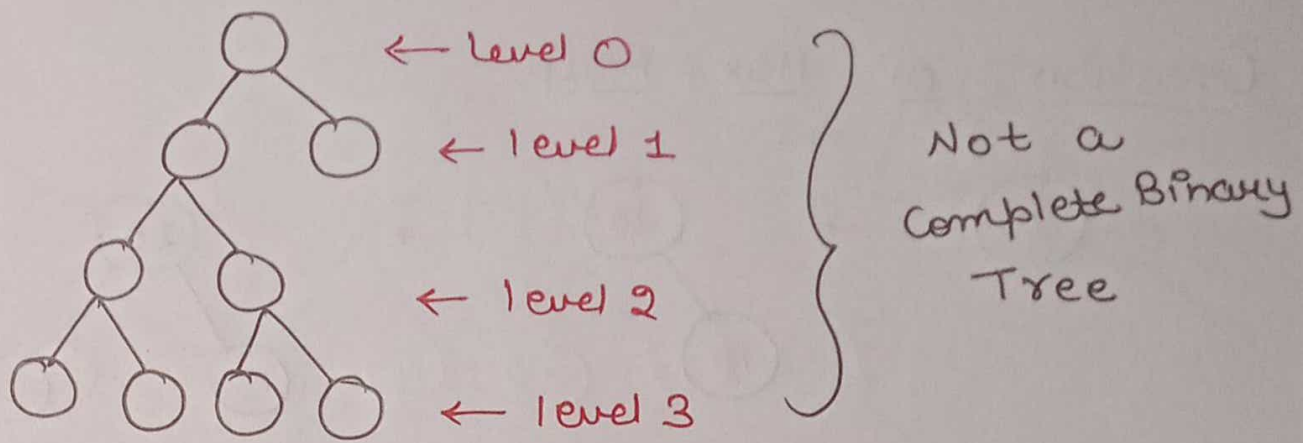
Example :



Example :



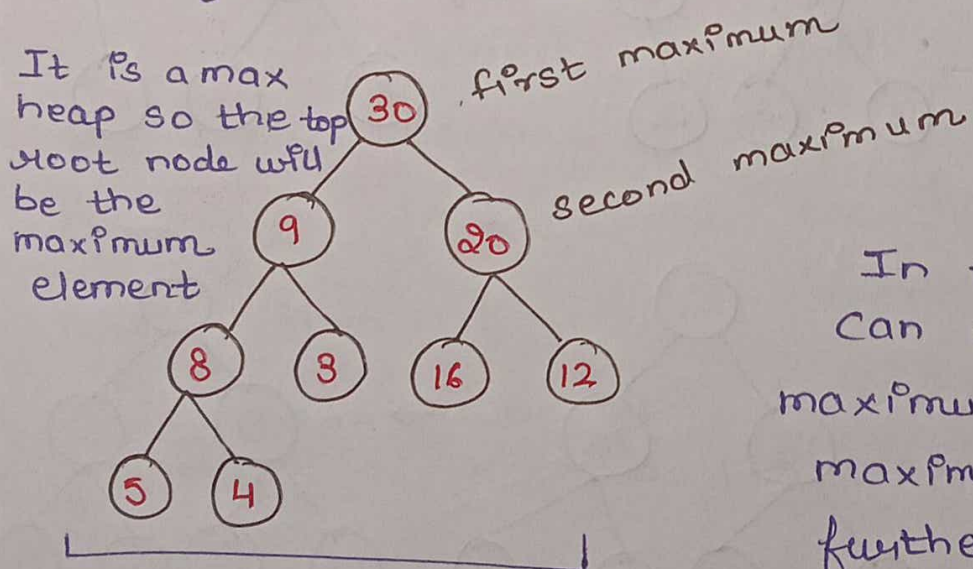
Not a CBT because at last level nodes should be filled from left side



Types of Heap

① Max heap

- CBT
- Parent node should be greater than or equal to child node
- It should be valid for all nodes

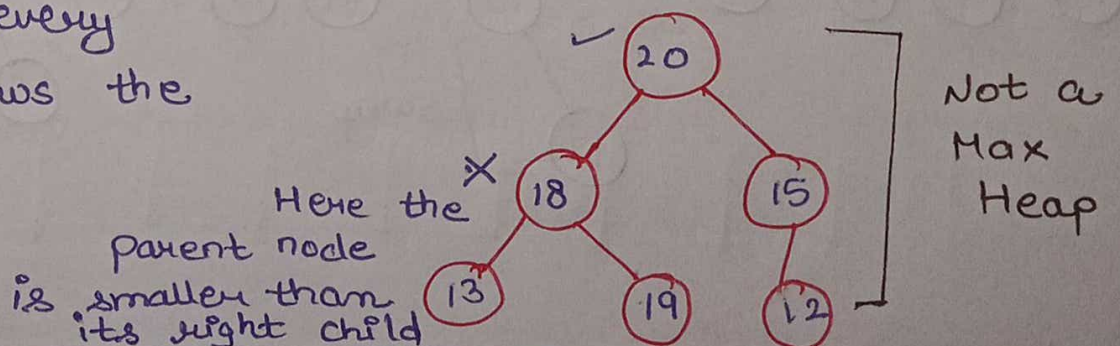


Max Heap every node follows the property

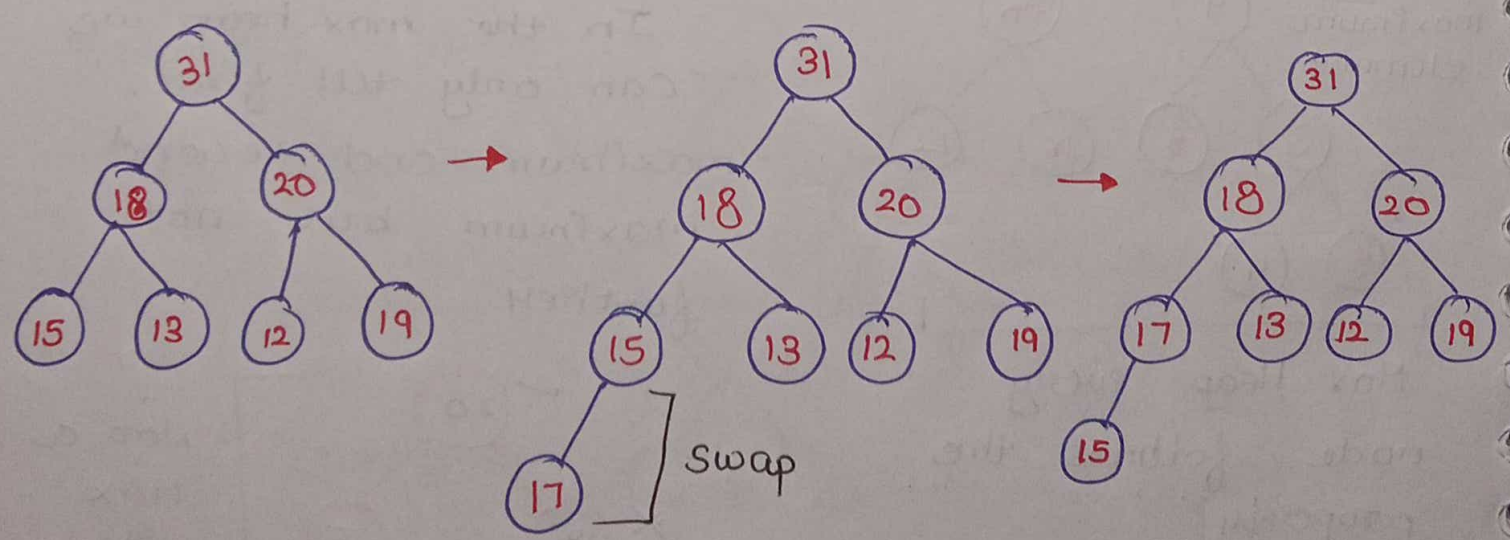
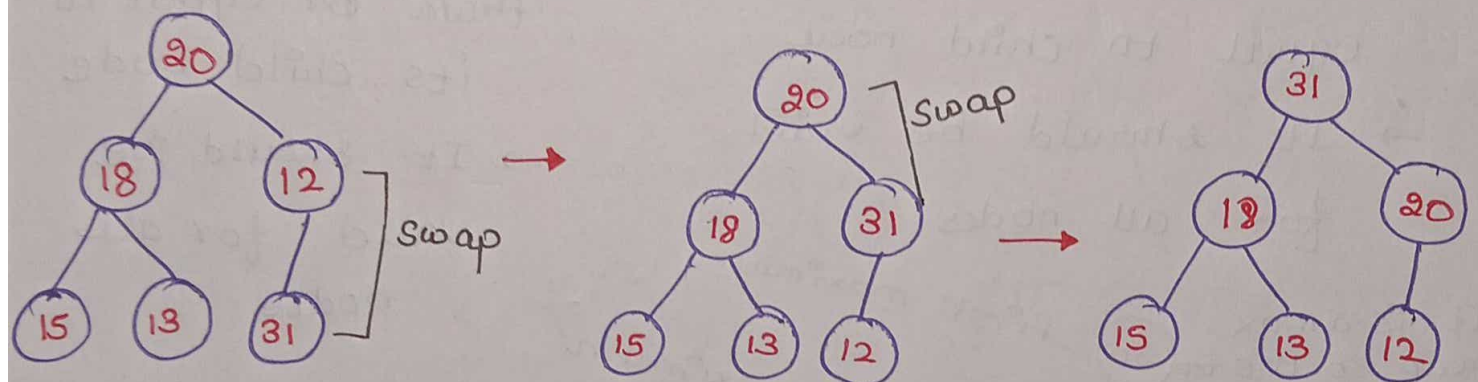
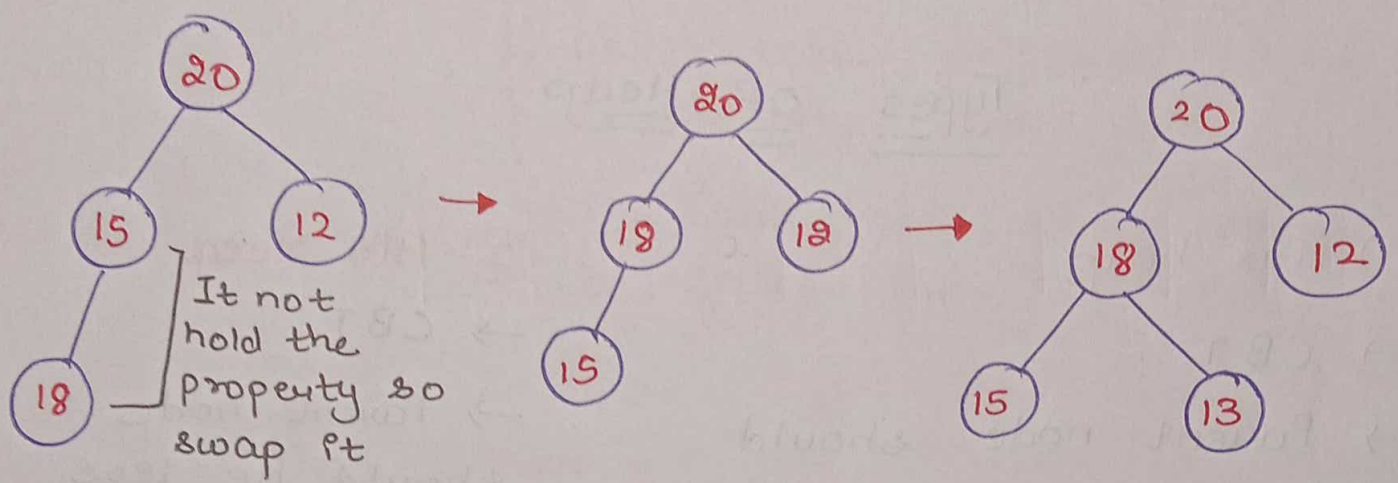
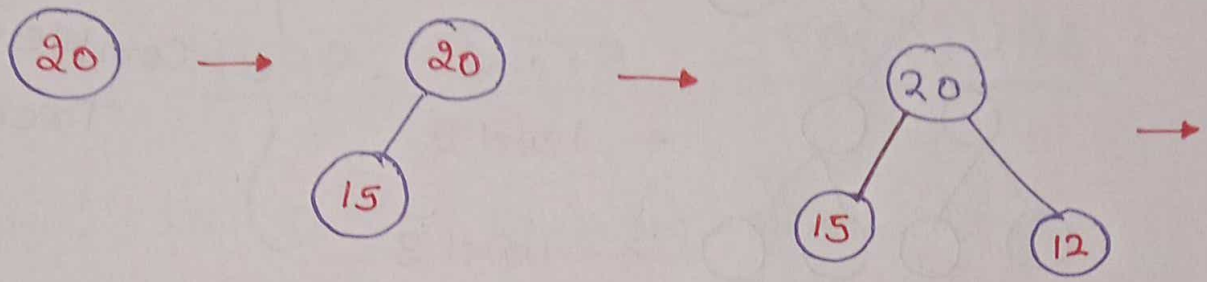
② Min Heap

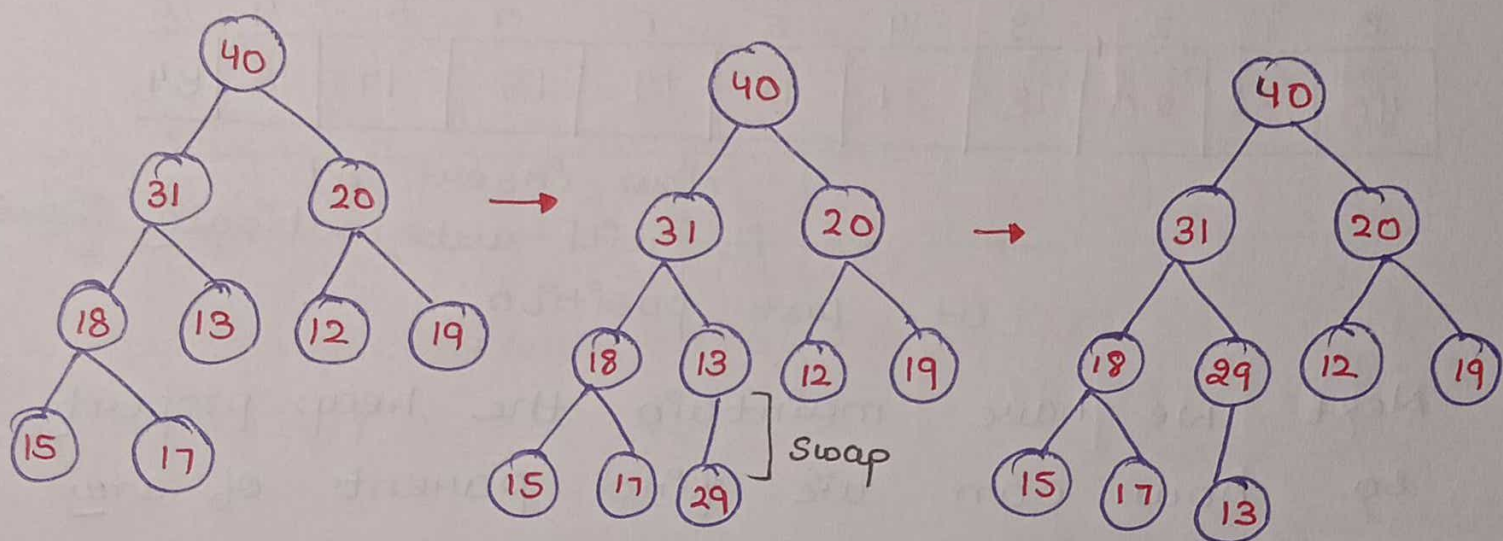
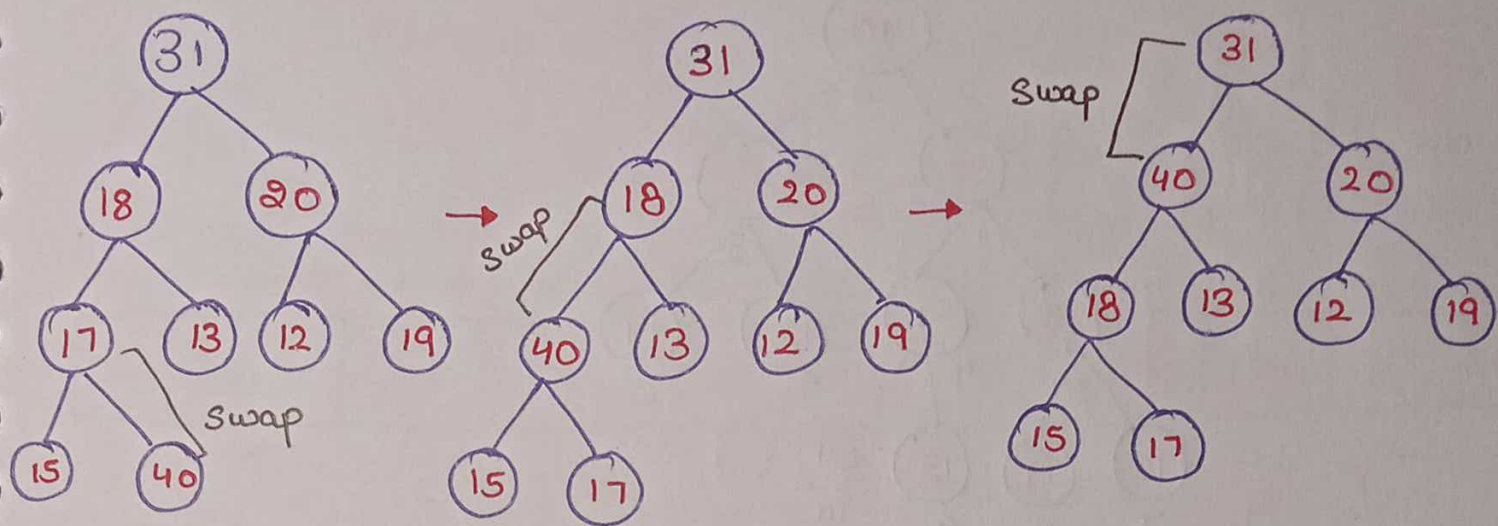
- CBT
- Parent node should be less than or equal to its child node
- It should be valid for all nodes

In the max heap we can only tell first maximum and second maximum but not further



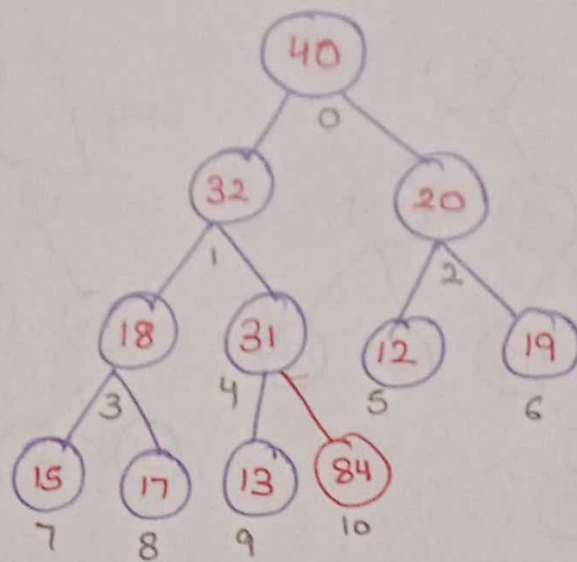
Creation of Max-Heap





If we implement it with Linked List then the code become messy because first we have to find where we have to insert the node then if it is greater than parent node then we have to swap it so it will take time and become complicated.

We implement it with Array because with array - index we can easily insert new element.



0	1	2	3	4	5	6	7	8	9	10
40	32	20	18	31	12	19	15	17	13	84

Now insert 84
so it will automatically insert
at last position

Next we have maintain the heap property
so how can we find parent of any
child node -

Parent - Index : i

$2 * i + 1$ (Left child)

$2 * i + 2$ (Right child)

Child - Index : i

Parent - Index : $\frac{i-1}{2}$

$$\boxed{10 \atop 84} \rightarrow \text{Parent} \therefore \frac{i-1}{2} = \frac{10-1}{2} = 4 \quad \boxed{4 \atop 31}$$

84 > 31 so it
will swap with 4th index

0	1	2	3	4	5	6	7	8	9	10
40	32	20	18	84	12	19	15	17	13	31

4
84

$$\text{Parent} = \frac{i-1}{2} = \frac{4-1}{2} = 1$$

1
32

Again $84 > 32$ so
swap it with 1st

0	1	2	3	4	5	6	7	8	9	10
40	84	20	18	32	12	19	15	17	13	31

1
84

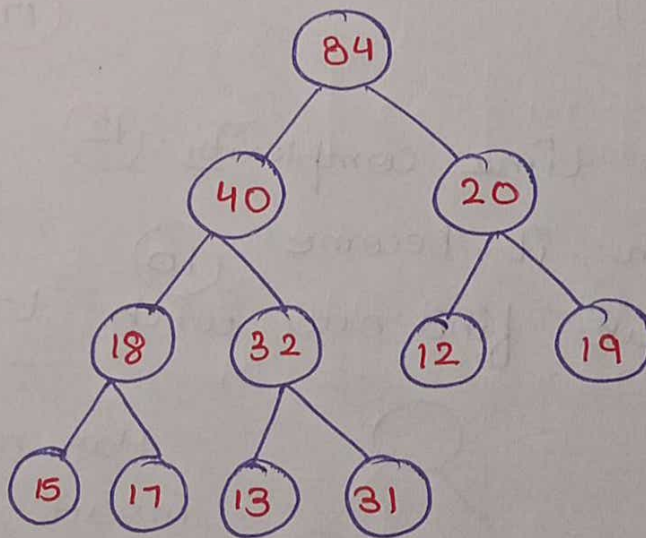
$$\text{Parent} = \frac{i-1}{2} = \frac{1-1}{2} = 0$$

0
40

$84 > 40$ swap

0	1	2	3	4	5	6	7	8	9	10
84	40	20	18	32	12	19	15	17	13	31

Now stop because its zero-index



Visual Representation

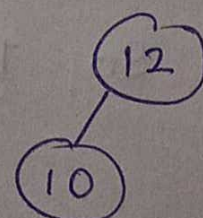
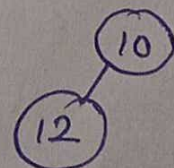
Time - Complexity

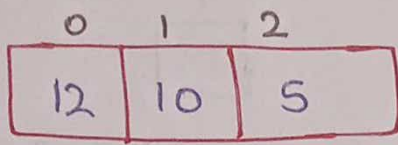
0	1
10	12

compare it
with parent

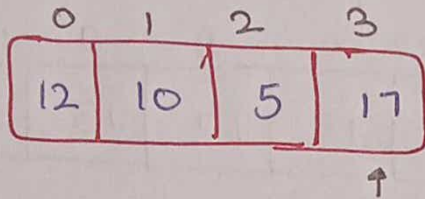
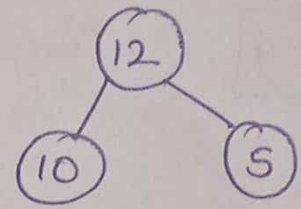
⇓

0	1
12	10





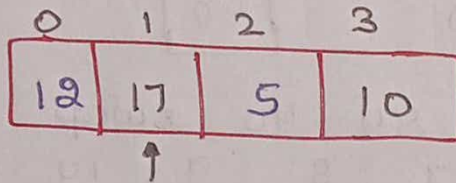
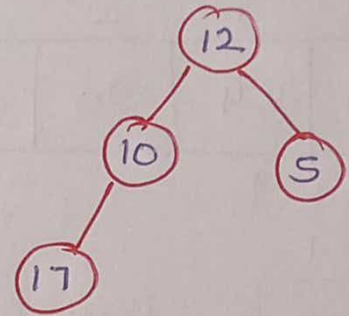
Comparing with
parent node
 $5 > 12 \times$



Parent - $\frac{i-1}{2}$

$$\frac{3-1}{2} = 1$$

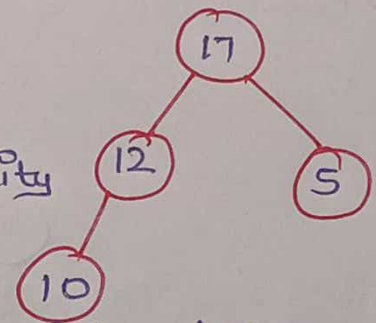
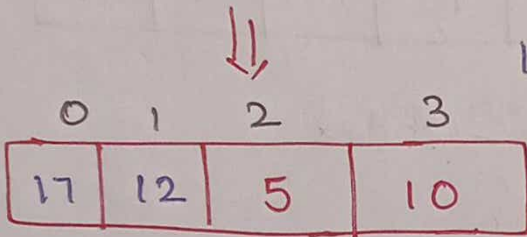
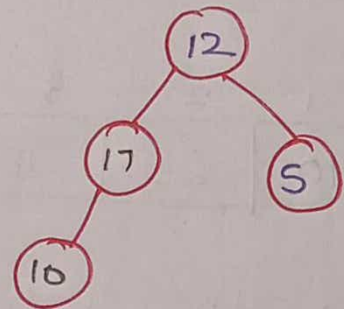
$17 > 10$ swap



Parent - $\frac{i-1}{2}$

$$\frac{1-1}{2} = 0$$

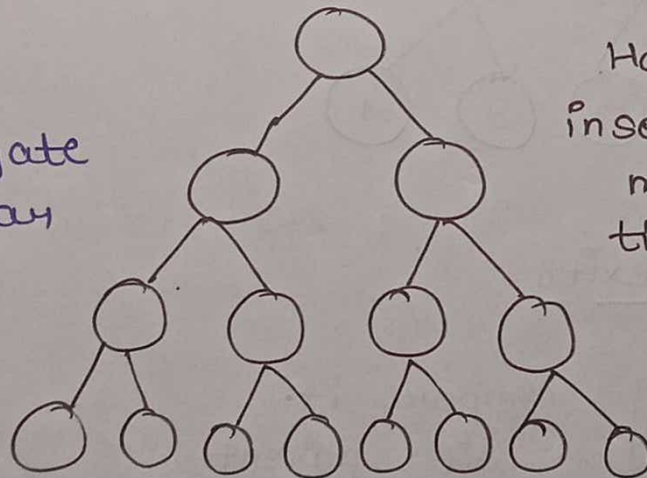
$17 > 12$ swap



If we find out time complexity
with array then it become
confusing so we find out with tree

Worst-Case :

last node propagate
karke sbse upar
jayega



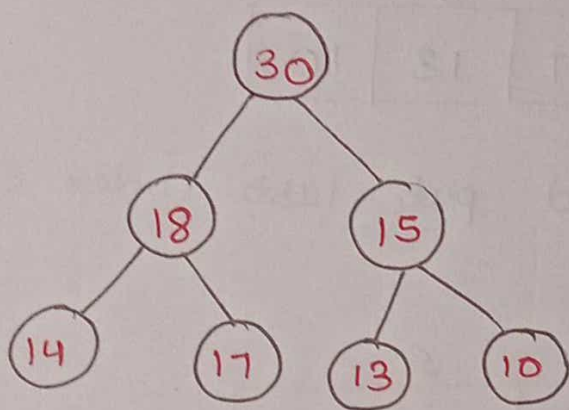
Har node ko
insert karne
mai - $\log n$
time lagega

If last level
 $(\frac{n}{2} + 1)$ node

So Time Complexity : $(\frac{n}{2} + 1) * \log n$

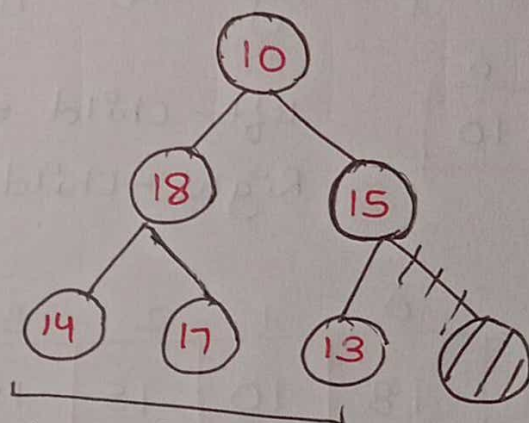
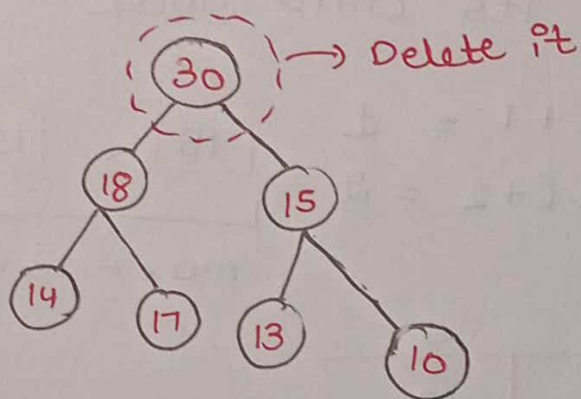
$O(n \log n)$

Delete a node from a Max Heap



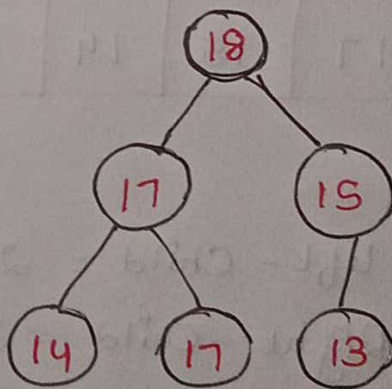
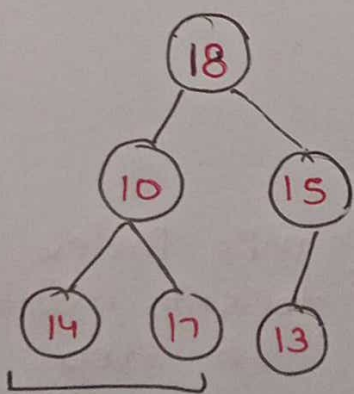
In Max-Heap we always delete the top not any other node

When we delete (30) we can't directly delete it we have to replace it with last node



Now it doesn't hold

property
Now the 10 will be
compare it with its children and ~~those~~ ^{that}
child will come up who is greater among
children



Inmai se voh
upar jayega jo
bada hoga

0	1	2	3	4	5	6
30	18	15	14	17	13	10

Delete 0th index and put last index element at 0th index

0	1	2	3	4	5
10	18	15	14	17	13

↑

Comparing it with its child node

0
10

$$\text{left-child} = 2 * i + 1 = 1$$

$$\text{Right-child} = 2 * i + 2 = 2$$

1 2
18 15

max - 18 at 1st

0	1	2	3	4	5
18	10	15	14	17	13

↑

1
10

$$\text{left-child} = 2 * i + 1 = 3$$

$$\text{Right-child} = 2 * i + 2 = 4$$

3 4
14 17

Replace it with 17

0	1	2	3	4	5
18	17	15	14	10	13

↑

4
10

$$\text{left-child} = 2 * 4 + 1 = 9$$

$$\text{Right-child} = 2 * 4 + 2 = 10$$

- This index doesn't exist so stop

$$T.C \Rightarrow O(\log n)$$



```
1  #include<iostream>
2  using namespace std;
3
4  class MaxHeap{
5
6      int *arr;
7      int size; // total elements in heap
8      int total_size; // total size of array
9
10     public:
11
12     MaxHeap(int n)
13     {
14         arr = new int[n];
15         size = 0;
16         total_size = n;
17     }
18
19     // Insertion onto heap
20
21     void insert(int value)
22     {
23         // EDGE CASE
24         if(size == total_size)
25         {
26             cout<<"Heap overflow"<<endl;
27             return;
28         }
29
30         arr[size] = value;
31         int index = size;
32         size++;
```



```
34 // inserting element to correct position
35 // Compare it with its parent
36
37 while (index > 0 && arr[(index-1)/2] < arr[index])
38 {
39     swap(arr[index], arr[(index-1)/2]);
40     index = (index-1)/2;
41 }
42
43 cout<<arr[index]<<" is inserted into the heap"<<endl;
44
45 }
46
47 void print()
48 {
49     for (int i = 0; i < size; i++)
50     {
51         cout<<arr[i]<<" ";
52     }
53
54     cout<<endl;
55
56 }
57
58 void Heapify (int index)
59 {
60     int largest = index;
61     int left = 2*index+1;
62     int right = 2*index+2;
63
```

```

64 // largest will store the index of the element which is greater between parent, left child, right child
65
66 if(left < size && arr[left] > arr[largest])
67 {
68     largest = left;
69 }
70
71 if(right < size && arr[right] > arr[largest])
72 {
73     largest = right;
74 }
75
76 if(largest != index)
77 {
78     swap(arr[index], arr[largest]);
79     Heapify(largest);
80 }
81 }
82
83 void Delete()
84 {
85     if(size == 0)
86     {
87         cout<<"Heap Underflow";
88         return;
89     }
90
91     cout<<arr[0]<<" deleted from the heap"<<endl;
92
93     //Replacing the last element with first
94     arr[0] = arr[size-1];
95     size--;

```



```
97         if(size == 0)
98             return;
99
100         // Heapify operation: Putting the elements to its correct position
101         Heapify(0);
102     }
103
104 };
105
106 int main()
107 {
108     MaxHeap H1(6);
109     H1.insert(4);
110     H1.insert(14);
111     H1.insert(11);
112
113     H1.Delete();
114
115     H1.print();
116
117     H1.insert(114);
118     H1.insert(24);
119     H1.insert(1);
120     H1.insert(10);
121
122     H1.print();
123
124     H1.Delete();
125     H1.Delete();
126     H1.Delete();
127
128     H1.print();
129
130 }
```